

The Build Journey

The Journey Story

---> give only the story part a try it's interesting (and short)

So before I even picked this problem, I actually had a completely different idea in mind.

I was thinking of building an edge / local AI voice payment agent. The whole concept was to create a conversational payment assistant with multilingual speech for people who struggle with conventional digital payment apps—especially older people, people who need accessibility options, or anyone who feels way more comfortable using their local language instead of clicking around confusing screens. The AI would listen to voice instructions, confirm the transaction, and guide the user through the payment step-by-step.

I was particularly interested in running the AI locally on-device (Edge AI). Running it on device would keep sensitive personal conversations and payment details from being sent over to servers unnecessarily, and it would also cut down latency.

So that was my plan. But then I saw that Razorpay was already heavily invested in voice-first and conversational commerce through their partnership with Sarvam, and they were already actively announcing it. Once I saw that, I thought of stepping back from it. There was really no point building something Razorpay was already doing and announcing.

So I stepped back to find a fresh problem.

And now, I didn't want to build just another generic finance app or an accounting dashboard that spits out balance sheets. That's boring, and honestly, merchants already have tools like Tally or QuickBooks for that.

I wanted to solve an actual friction point inside payment gateways. I kept thinking about how people actually interact with gateway dashboards like Razorpay. If you've ever run an online business or talked to a merchant (business where there is no dedicated tech finance managing person), you know the feeling: you log in, and you can see charts, total volumes, and success rate percentages like "82.4%".

On paper, 82% sounds okay. But behind the scenes, that missing 17.6% is thousands or even lakhs of rupees just vanishing into the void. When a transaction fails with strings like `GATEWAY_ERROR` or `PAYMENT_TIMED_OUT`, what is a merchant actually supposed to do with that? Do they call the customer? Do they retry? Is the bank down? Or did the customer just abandon the cart?

No dashboard answers the real questions merchants actually care about:

- How much money am I actually losing right now?
- How much of that lost money can I realistically get back?
- What exact button do I press or action do I take first to recover it?

To make sure this wasn't just my own random hypothesis, I spent time researching where merchants actually vent: Reddit (threads on r/developersIndia, r/ecommerce, r/smallbusiness), Quora, and used AI power

(perplexity to research abt the same across twitter and other places)

I could see the stories now:

1. "How much revenue are you losing to failed Razorpay payments without knowing it?" - IndianStartups (Mar 24, 2026)
https://www.reddit.com/r/indianstartups/comments/1s2ro4n/how_much_revenue_are_you_losing_to_failed/
Directly about invisible losses from failed payments and retry behavior.
2. "How do you actually track revenue lost from payment failures or webhook issues?" - r/SaaS (Jan 30, 2026)
https://www.reddit.com/r/SaaS/comments/1qrc3tb/how_do_you_actually_track_revenue_lost_from/
Focuses on lost revenue from failures, retries, and reconciliation gaps rather than just "missing payments."
3. Rackz - "Failed payments don't announce themselves. They just quietly eat 5-10% of your MRR" - @rackzai (profile thread, 2025-2026)
<https://x.com/rackzai>
Describes failed payments as silent revenue loss and highlights detection of failures, renewal risks, and checkout issues
4. "Anyone else annoyed that Razorpay's dashboard doesn't show you what you actually keep?" - r/Razorpay (Apr 4, 2026)
https://www.reddit.com/r/Razorpay/comments/1scahpv/anyone_else_annoyed_that_razorpays_dashboard/
Merchants frustrated that fees, GST, failed payments, and refunds make the "real" amount unclear.

That was the "aha!" moment for me. Razorpay already records the exact failure reasons, error steps, and transaction metadata. The raw data exists. But it sits in isolated rows. It doesn't guide the merchant on what to do.

Once I decided on this problem statement, I also did a thorough research on how Razorpay currently handles merchants. I looked into the webhooks part, the redirection of payment gateway vs the in-app checkout screen, the portals, and all those things. I wanted to understand how Razorpay actually works in these parts because I didn't want to build a separate SaaS application. I wanted to build an integratable extension for current Razorpay's platform.

I wanted to build an active enhancement layer directly on top of Razorpay. Not a dashboard that just shows you more graphs, but a co-pilot that scans the leaks, quantifies them in actual rupees (₹), and hands you a ready-to-execute "Recovery Play." I started late, had very less time but started thinking I'll pull off howmuch ever i can and add rest to the "future scope" haha.. (not bad right?)

The "Anti-Gimmick" AI Philosophy

I made a rule early on: no AI math. In fintech, if an LLM hallucinates an extra zero, you lose the merchant's trust forever. So I built a strictly deterministic foundation. Python, pandas, and statistical z-scores do the heavy lifting to find the real money. The AI acts purely as a strategist—it takes those rock-solid numbers and translates them into plain English so you don't need to be a payment engineer to know what to do.

How I Built It (The Short Version)

Phases 1-3: Building the Brain

Before anything else, I needed raw truth. I built a deterministic engine that separates terminal failures (like insufficient funds) from actual recoverable errors (like bank timeouts). Then, instead of dumb threshold alerts, I wired up a rolling Z-score anomaly detector to spot silent gateway outages.

But finding a leak isn't enough—you need to know what to fix first. So I wrote a scoring system that projects exactly how many rupees each fix will yield using time-series forecasting. The result? A perfectly ranked list of "Recovery Plays" based on pure math.

Test outputs: [Phase 2 Test](#), [Batch Eval 1](#), [Batch Eval 2](#)

Phases 4-5: Giving it a Face

Next, I built the **Grounded AI Copilot**. It reads the pre-computed facts and answers questions without hallucinating. But an advisor without execution is just a chat widget. So I built the **Gated Action Gateway**—an idempotent, secure execution layer that lets the merchant click "Run Play" and safely retry payments or contest disputes. Finally, I wrapped it all in a sleek, Razorpay-inspired UI with electric blues and glassmorphism.

Test outputs: [Phase 4 Terminal Tests](#), [Frontend UI Final Result](#)

Phases 6 & 7: The Agentic Evolution (Where it gets really cool)

Up to this point, the system was smart but passive. It only moved when you clicked a button. I wanted it to act and learn autonomously. So, I added three agents to complete the vision:

1. **The Feedback Agent (System Memory):** Every time a play executes, it compares the actual recovery against its internal assumptions. If there's a gap, it auto-recalibrates the engine. It literally gets smarter over time.
2. **The Reactive Agent (Real-Time Action):** This intercepts live webhooks. If a payment fails under ₹5,000 and the confidence is high, it doesn't wait for you—it just auto-executes the retry instantly. For bigger amounts, it drafts a recommendation. It even has a built-in circuit breaker to halt retries if a bank goes completely down.
3. **Conversational Policy Control:** I wanted merchants to control these agents without digging through confusing settings. Now, you just tell the Copilot: "Turn off auto-retry for card failures." The system

parses your intent deterministically and updates the agent's policy in real-time. The AI doesn't decide the rules; it just sets the dials.

And that's how RazorRecover evolved from a smart analytics dashboard into a fully autonomous revenue engine.